

SIMATS ENGINEERING

ASSESSMENT TOOL 1

COURSE NAME: INTERNET PROGRAMMING
COURSE CODE: CSA4305

COURSE OUTCOME COVERED

CO 3: Implement dynamic web applications by utilizing DOM-based client-side scripting and employing Java Servlets for server-side request processing and response handling. (L3)

Assessment Tool : Scenario-Based Assessment

Weightage: 40%

QUESTIONS:

Total Marks: 30

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet. Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication. Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Question 3

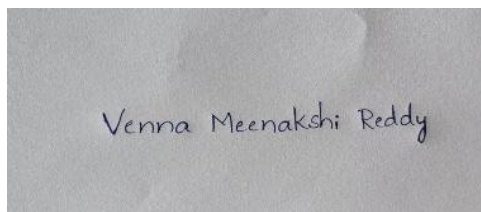
Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

Code of Conduct:

/ VENNA MEENAKSHI REDDY(192471048) (Name / Reg No) certify that this submission is my original work and that I have adhered to the guidelines specified for this assessment. I understand that any violation of academic integrity rules will result in disciplinary action.

Signature of the student:



RUBRICS FOR CALCULATION:

Criteria	Weightage (%)	Level 4 (Exemplary)	Level 3 (Proficient)	Level 2 (Developing)	Level 1 (Beginning)
Understanding of Scenario	20%	Demonstrates clear and complete understanding of the scenario and context	Shows good understanding with minor gaps	Partial understanding; misses key aspects	Misinterprets or fails to understand the scenario
Identification of Key Issues	20%	Accurately identifies all relevant issues and factors involved	Identifies most key issues with minor omissions	Identifies limited issues; some irrelevant points	Unable to identify key issues
Application of Concepts	25%	Applies appropriate concepts effectively to address the scenario	Applies concepts with minor errors	Limited or partially correct application	Unable to apply relevant concepts
Decision Making	20%	Makes well-reasoned, logical decisions with strong rationale	Makes reasonable decisions with some justification	Makes weak decisions with limited justification	No clear decisions made or rationale provided
Clarity of Response	15%	Response is clear, structured, and logically presented	Generally clear with minor issues	Somewhat unclear or poorly structured	Disorganized and difficult to understand

Question 1

Suppose that a college wants to develop an online student registration system where students submit their details through a web form, and the data is processed using a Java Servlet.

Explain the architecture of a servlet-based application. Design and implement a servlet to handle form data using both GET and POST methods. Describe the servlet life cycle (init, service, destroy) and justify which method is more appropriate for handling sensitive data.

Servlet-Based Online Student Registration System

1. Understanding of the Scenario

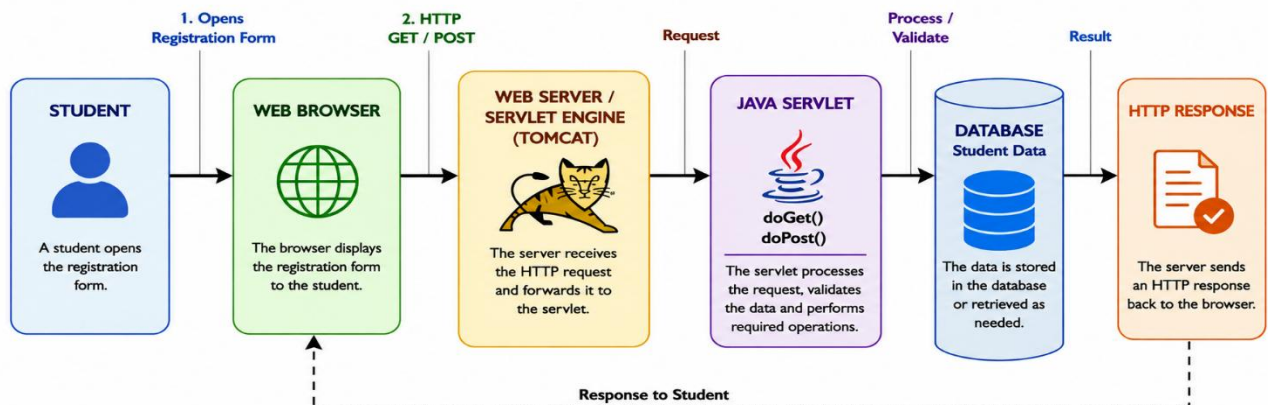
A college wants to develop an online student registration system in which students enter their details through a web form. The submitted data is sent to a Java Servlet, which processes the request and generates an appropriate response.

The system should support both GET and POST methods. However, because student registration may contain sensitive information such as mobile number, email, address, or other personal details, the POST method is more appropriate for actual registration.

2. Architecture of a Servlet-Based Application

A servlet-based application follows a client–server architecture.

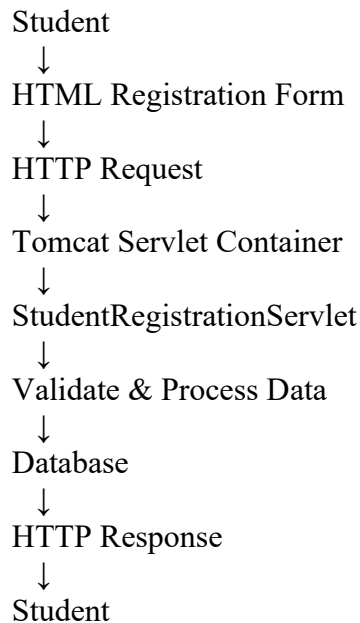
Architecture



Main Components

1. **Client/Browser** – Student enters registration information.
2. **HTML Form** – Collects details such as name, register number, email and department.
3. **Web Server/Servlet Container** – Apache Tomcat receives the HTTP request.
4. **Servlet** – Processes the submitted form data using `doGet()` or `doPost()`.
5. **Database** – Stores student information after validation.
6. **HTTP Response** – Servlet sends confirmation or error information back to the browser.

Request Flow



This architecture separates the presentation layer (HTML) from the processing layer (Servlet) and the data layer (Database), making the application easier to maintain.

3. HTML Registration Form

The following form can send data using either GET or POST.

register.html

```
<!DOCTYPE html>
<html>
<head>
  <title>Student Registration</title>
</head>
<body>

<h2>Student Registration Form</h2>

<form action="register" method="post">

  Register Number:
  <input type="text" name="regno"><br><br>

  Student Name:
  <input type="text" name="name"><br><br>

  Email:
  <input type="email" name="email"><br><br>

  Department:
  <input type="text" name="department"><br><br>

  Mobile:
  <input type="text" name="mobile"><br><br>

  <input type="submit" value="Register">
```

```
</form>
```

```
</body>
```

```
</html>
```

Here:

```
method="post"
```

means the browser sends the form data using the HTTP POST method.

To test GET, it can be changed to:

```
method="get"
```

4. Servlet Implementation Using GET and POST

StudentRegistrationServlet.java

```
import java.io.*;
```

```
import jakarta.servlet.*;
```

```
import jakarta.servlet.http.*;
```

```
public class StudentRegistrationServlet extends HttpServlet {
```

```
    // Called only once when servlet is created
```

```
    public void init() throws ServletException {
```

```
        System.out.println("Servlet initialized");
```

```
    }
```

```
    // Handles GET requests
```

```
    protected void doGet(HttpServletRequest request,
```

```
                          HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```
        String name = request.getParameter("name");
```

```
        String regno = request.getParameter("regno");
```

```
        out.println("<html>");
```

```
        out.println("<body>");
```

```
        out.println("<h2>GET Request</h2>");
```

```
        out.println("<p>Register Number: " + regno + "</p>");
```

```
        out.println("<p>Student Name: " + name + "</p>");
```

```
        out.println("</body>");
```

```
        out.println("</html>");
```

```
    }
```

```
    // Handles POST requests
```

```
    protected void doPost(HttpServletRequest request,
```

```
                          HttpServletResponse response)
```

```
        throws ServletException, IOException {
```

```
        response.setContentType("text/html");
```

```
        PrintWriter out = response.getWriter();
```

```

String regno = request.getParameter("regno");
String name = request.getParameter("name");
String email = request.getParameter("email");
String department = request.getParameter("department");
String mobile = request.getParameter("mobile");

// Basic validation
if (regno == null || name == null ||
    email == null || department == null ||
    mobile == null ||
    regno.isEmpty() || name.isEmpty()) {

    out.println("<h2>Registration Failed</h2>");
    out.println("<p>Please fill all required fields.</p>");
    return;
}

out.println("<html>");
out.println("<body>");
out.println("<h2>Student Registration Successful</h2>");
out.println("<p>Register Number: " + regno + "</p>");
out.println("<p>Name: " + name + "</p>");
out.println("<p>Email: " + email + "</p>");
out.println("<p>Department: " + department + "</p>");
out.println("<p>Mobile: " + mobile + "</p>");
out.println("</body>");
out.println("</html>");
}

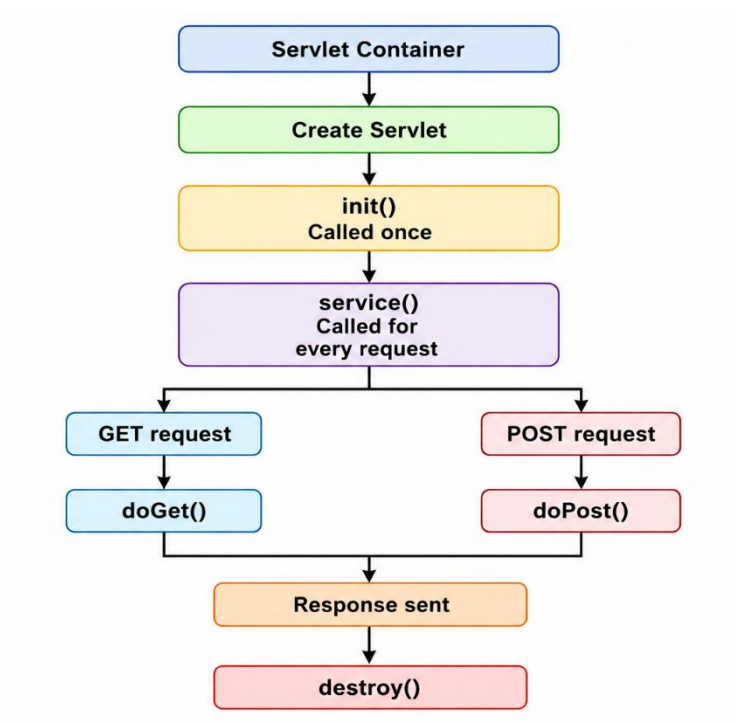
// Called when servlet is removed
public void destroy() {
    System.out.println("Servlet destroyed");
}
}

```

If using Tomcat 9 or earlier, replace jakarta.servlet.* with javax.servlet.*.

5. Servlet Life Cycle

The servlet life cycle consists mainly of three important methods:



a) init()

```
public void init() {  
    System.out.println("Servlet initialized");  
}
```

- Called only once when the servlet is created.
- Used for initialization tasks.
- Example: establishing resources or reading configuration.

Decision

Initialization should be performed in `init()` rather than repeatedly for every request because it improves efficiency.

b) service()

The servlet container calls the `service()` method whenever a request arrives.

For an `HttpServlet`, `service()` determines the HTTP request type and internally calls the appropriate method:

GET → `doGet()`

POST → `doPost()`

For example:

```
protected void doGet(...) {  
    // Process GET request  
}
```

```
protected void doPost(...) {  
    // Process POST request  
}
```

Thus, the programmer normally overrides `doGet()` and `doPost()` instead of directly overriding `service()`.

c) destroy()

```
public void destroy() {  
    System.out.println("Servlet destroyed");  
}
```

- Called once before the servlet is removed from service.
- Used for cleanup activities.
- Example: closing resources or releasing connections.

6. GET vs POST

Feature	GET	POST
Data location	URL	Request body
Visibility	Data visible in URL	Not normally displayed in URL
Data size	Limited	Suitable for larger data
Bookmarking	Possible	Not normally used
Suitable for	Searching/retrieving data	Form submission
Sensitive information	Not recommended	More appropriate
Example	Search request	Student registration

7. Why POST is More Appropriate for Sensitive Student Data

For the student registration system, POST should be selected.

Reasons:

1.Data is not placed in the URL

GET places form parameters directly in the URL, such as:

/register?name=John&mobile=9876543210

This can expose personal information through browser history, bookmarks, logs, or copied URLs.

2. POST sends data in the request body

The registration details are transmitted in the HTTP request body instead of being appended to the URL.

3. Better suited for registration

Student registration is an operation that submits and potentially creates data, so POST semantically fits the operation better than GET.

4. Better privacy

POST avoids exposing personal details directly in the URL. However, POST alone does not encrypt data.

5. HTTPS is still required

For genuinely sensitive information, the system should use:

POST + HTTPS

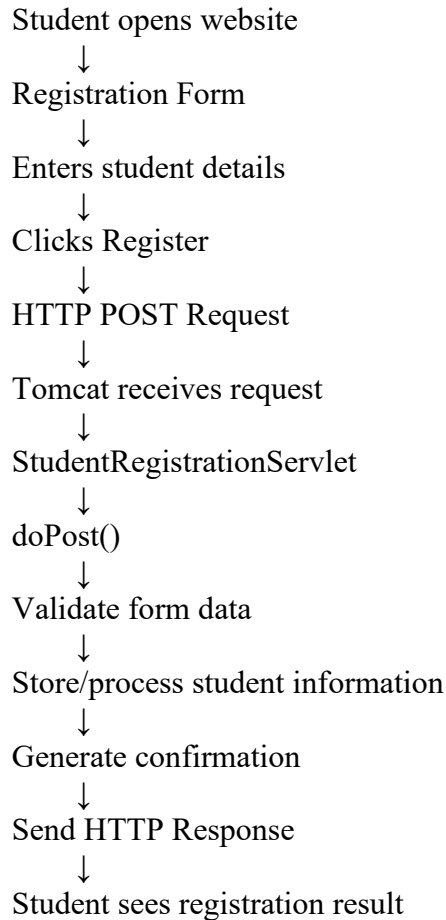
HTTPS encrypts the communication between the student's browser and the server.

Important Decision

Therefore:

GET is suitable for retrieving/searching non-sensitive information, whereas POST is the better choice for submitting student registration details. For real security, POST must be combined with HTTPS, server-side validation, authentication/authorization where needed, and secure database handling.

8. Overall Working of the Proposed System



9. Analysis and Justification

The proposed design demonstrates a clear understanding of the scenario by separating user interface, request processing, and data storage. The servlet uses doGet() for retrieving/requesting non-sensitive information and doPost() for registration submission.

The key design decision is to use POST for student registration, because registration changes or creates server-side data and may contain personal information. GET is therefore retained for appropriate retrieval operations rather than being used for sensitive submissions. The servlet life cycle is also considered: init() performs one-time initialization, service() dispatches incoming requests to doGet() or doPost(), and destroy() performs cleanup.

Question 2

Suppose that a website requires users to log in and maintain their session across multiple pages after authentication.

Design a session management system using HttpSession and Cookies in Servlets. Explain how session tracking works in web applications and compare session-based tracking with cookies.

Session Management Using HttpSession and Cookies in Servlets

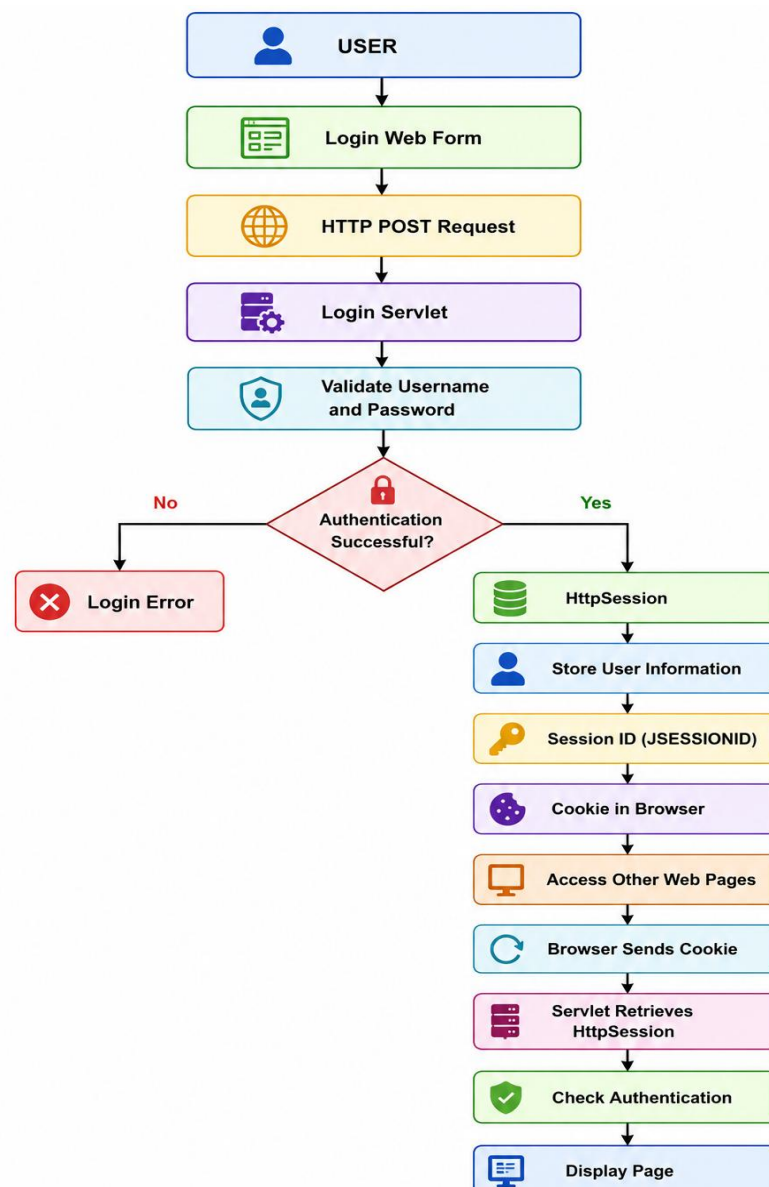
1. Understanding of the Scenario

A website requires users to log in once and access multiple pages without logging in again. After successful authentication, the application must remember the user's identity across different HTTP requests.

Since HTTP is stateless, each request is treated independently. Therefore, a session-tracking mechanism is required to associate multiple requests with the same authenticated user.

A suitable design is to use HttpSession for server-side session management and Cookies to maintain the session identifier between the browser and server.

2. Session Management Architecture



Main components

1. **Login Form** – collects username and password.
2. **Login Servlet** – validates the user's credentials.
3. **HttpSession** – stores authenticated user information on the server.
4. **Cookie** – normally stores the session identifier, such as JSESSIONID, in the browser.
5. **Protected Servlets/Pages** – retrieve the session and verify that the user is authenticated.
6. **Logout Servlet** – invalidates the session and removes the authenticated state.

3. Implementation Using HttpSession

Login HTML Form

```
<!DOCTYPE html>
<html>
<head>
  <title>Login</title>
</head>
<body>

<h2>User Login</h2>

<form action="login" method="post">
  Username:
  <input type="text" name="username" required><br><br>

  Password:
  <input type="password" name="password" required><br><br>

  <input type="submit" value="Login">
</form>

</body>
</html>
```

Login Servlet

```
import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LoginServlet extends HttpServlet {

    protected void doPost(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        String username = request.getParameter("username");
        String password = request.getParameter("password");

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        // Simple authentication for demonstration
        if (username.equals("admin") && password.equals("1234")) {

            // Create session
```

```

        HttpSession session = request.getSession();

        // Store user information in session
        session.setAttribute("username", username);

        out.println("<h2>Login Successful</h2>");
        out.println("<p>Welcome " + username + "</p>");
        out.println("<a href='home'>Go to Home Page</a>");

    } else {

        out.println("<h2>Invalid Username or Password</h2>");
        out.println("<a href='login.html'>Try Again</a>");

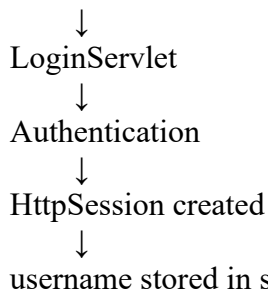
    }
}
}

```

What happens here?

After successful login:

Username + Password



The server associates the session with a unique session ID, commonly sent to the browser through the JSESSIONID cookie.

4. Accessing the Session on Another Page

For example, a user visits the Home page after logging in.

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class HomeServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                        HttpServletResponse response)
        throws ServletException, IOException {

        response.setContentType("text/html");
        PrintWriter out = response.getWriter();

        HttpSession session = request.getSession(false);

        if (session != null &&
            session.getAttribute("username") != null) {

            String username =
                (String) session.getAttribute("username");

            out.println("<h2>Home Page</h2>");

```

```

        out.println("<p>Welcome, " + username + "</p>");
        out.println("<p>You are logged in.</p>");
        out.println("<a href='logout'>Logout</a>");

    } else {

        out.println("<h2>Access Denied</h2>");
        out.println("<p>Please login first.</p>");
    }
}
}

```

Why getSession(false)?

request.getSession(false);

means:

- Return the existing session if one exists.
- Do not create a new session if the user is not logged in.

This is useful for protected pages.

5. Logout Servlet

```

import java.io.*;
import jakarta.servlet.*;
import jakarta.servlet.http.*;

public class LogoutServlet extends HttpServlet {

    protected void doGet(HttpServletRequest request,
                          HttpServletResponse response)
        throws ServletException, IOException {

        HttpSession session = request.getSession(false);

        if (session != null) {
            session.invalidate();
        }

        response.sendRedirect("login.html");
    }
}

```

When the user clicks Logout:

Logout Request



LogoutServlet



session.invalidate()



Session destroyed



User must login again

6. Using Cookies for Session Tracking

A Cookie is a small piece of data stored in the user's browser.

Example:

```
Cookie userCookie =
```

```
    new Cookie("username", "admin");
```

```
userCookie.setMaxAge(60 * 60);
```

```
response.addCookie(userCookie);
```

The browser stores the cookie and sends it with subsequent requests.

To read it:

```
Cookie[] cookies = request.getCookies();
```

```
if (cookies != null) {
```

```
    for (Cookie cookie : cookies) {
```

```
        if (cookie.getName().equals("username")) {
```

```
            String username = cookie.getValue();
```

```
            System.out.println("User: " + username);
```

```
        }
```

```
    }
```

```
}
```

Important distinction

For a login application, do not store the user's password in a cookie.

A better approach is:

Browser



JSESSIONID Cookie



Server



HttpSession



User Information

The actual session data remains on the server, while the browser generally carries only the session identifier.

7. How Session Tracking Works

The complete process is:

Step 1 – Login

The user enters:

Username: admin

Password: 1234

Step 2 – Authentication

The Login Servlet validates the credentials.

Step 3 – Create Session

```
HttpSession session = request.getSession();
```

```
session.setAttribute("username", username);
```

Step 4 – Session ID

The servlet container creates a unique session ID such as:

JSESSIONID = ABC123XYZ

The ID is normally maintained through a cookie.

Step 5 – Subsequent Request

When the user opens another page, the browser sends:

JSESSIONID = ABC123XYZ

Step 6 – Server Finds Session

Tomcat uses the session ID to locate the corresponding HttpSession.

Step 7 – Authentication Check

The servlet checks:

```
session.getAttribute("username");
```

If the value exists, the user is considered authenticated.

Step 8 – Logout

The session is invalidated:

```
session.invalidate();
```

Therefore, the user can no longer access protected pages using that session.

8. HttpSession vs Cookies

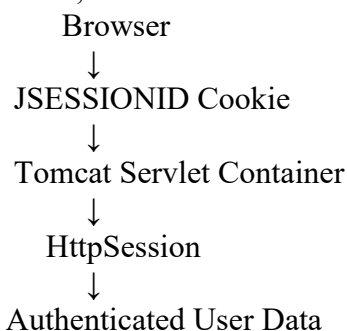
Feature	HttpSession	Cookies
Storage	Mainly server-side	Client/browser-side
Data exposure	Session data is not directly stored in browser	Cookie data is stored in browser
Identification	Uses a session ID	Uses cookie name/value
Security	Better for sensitive session information	Should not contain passwords or sensitive data
Lifetime	Can expire based on session timeout	Can be session-based or persistent
Server memory	Uses server resources	Requires less server-side storage
Typical use	Login/session management	Preferences, identifiers, tracking
Java Servlet support	Directly supported by HttpSession	Supported by Cookie class

9. Decision and Justification

For this login system, HttpSession is the more appropriate primary mechanism because authentication information should be maintained securely on the server.

Cookies can still be used as part of the mechanism because the servlet container commonly uses a JSESSIONID cookie to identify the user's HttpSession.

Therefore, the recommended design is:



This design avoids storing sensitive information such as passwords directly in cookies and provides a clear separation between the client-side session identifier and server-side session data.

10. Conclusion

A servlet-based web application is stateless by default, so session tracking is necessary when a user must remain logged in across multiple pages. HttpSession provides server-side session management, while cookies can carry the session identifier between requests. For an authenticated website, using HttpSession together with a secure session cookie is the most appropriate approach because it maintains user state while avoiding storage of sensitive authentication data in the browser.

Question 3

Suppose that an e-commerce platform needs to store and retrieve product details from a database using a Java-based web application.

Design a JDBC-based application to connect to a database, insert product data, and retrieve it using SQL queries. Explain the JDBC architecture and describe the steps involved in database connectivity along with proper exception handling.

JDBC-Based E-Commerce Product Management System

1. Understanding the Scenario

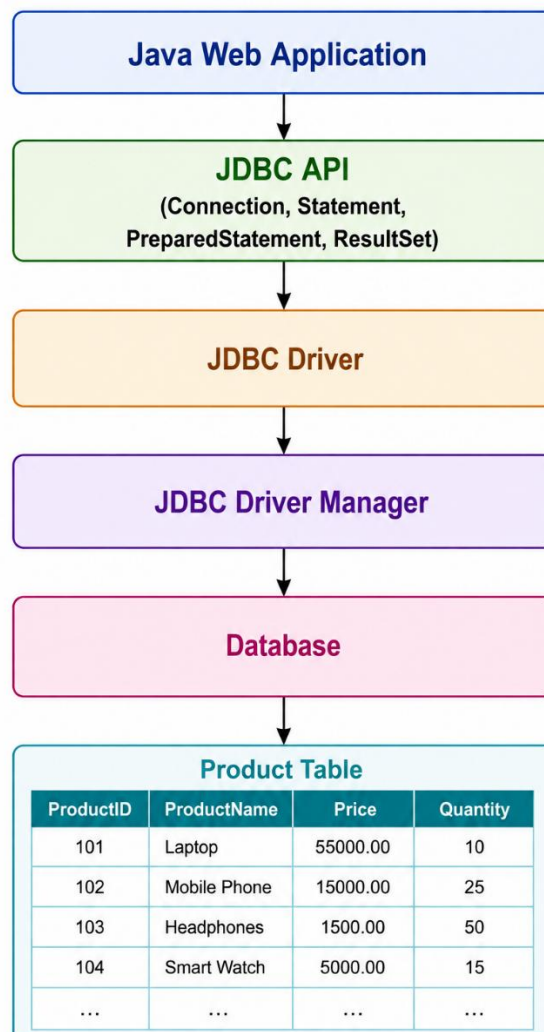
An e-commerce platform needs to store and retrieve product details using a Java-based web application. JDBC (Java Database Connectivity) provides a standard way for a Java application to communicate with a relational database.

The proposed application will:

- Connect the Java application to a database.
- Insert product details into a Product table.
- Retrieve product details using an SQL SELECT query.
- Handle database errors using proper exception handling.
- Close database resources safely after use.

2. JDBC Architecture

The JDBC architecture acts as a bridge between the Java application and the database.



Components

1. Java Application

The web application contains Java code that needs to store or retrieve product information.

2. JDBC API

Provides classes and interfaces such as:

- Connection
- PreparedStatement
- Statement
- ResultSet
- SQLException

3. JDBC Driver

The driver translates JDBC calls into database-specific commands.

For example, a MySQL database uses the MySQL JDBC driver.

4. Database

Stores the product information permanently.

Example:

Product(ProductID, ProductName, Price, Quantity)

3. JDBC Request Flow

User



Java Web Application



JDBC API



JDBC Driver



Database



SQL Query



Database Result



Java Application



Web Response

This architecture separates the application logic from the database communication mechanism.

4. Database and Table Creation

For example, using MySQL:

```
CREATE DATABASE ecommerce;
```

```
USE ecommerce;
```

```
CREATE TABLE Product (  
    ProductID INT PRIMARY KEY,  
    ProductName VARCHAR(100),  
    Price DECIMAL(10,2),  
    Quantity INT  
);
```

5. JDBC-Based Java Program

The following program demonstrates both inserting and retrieving product data.

```
import java.sql.*;

public class ProductJDBC {

    static final String URL =
        "jdbc:mysql://localhost:3306/ecommerce";

    static final String USER = "root";
    static final String PASSWORD = "root";

    public static void main(String[] args) {

        // Insert product
        insertProduct(101, "Laptop", 55000.00, 10);

        // Retrieve products
        displayProducts();
    }

    // Method to insert product
    public static void insertProduct(
        int id,
        String name,
        double price,
        int quantity) {

        String sql =
            "INSERT INTO Product " +
            "(ProductID, ProductName, Price, Quantity) " +
            "VALUES (?, ?, ?, ?)";

        try (
            Connection con =
                DriverManager.getConnection(URL, USER, PASSWORD);

            PreparedStatement ps =
                con.prepareStatement(sql)
        ) {

            ps.setInt(1, id);
            ps.setString(2, name);
            ps.setDouble(3, price);
            ps.setInt(4, quantity);

            int rows = ps.executeUpdate();

            System.out.println(
                rows + " product inserted successfully."
            );

        } catch (SQLException e) {
```

```

        System.out.println(
            "Database error while inserting product."
        );

        System.out.println(
            "Error: " + e.getMessage()
        );
    }
}

// Method to retrieve products
public static void displayProducts() {

    String sql =
        "SELECT ProductID, ProductName, Price, Quantity " +
        "FROM Product";

    try (
        Connection con =
            DriverManager.getConnection(URL, USER, PASSWORD);

        PreparedStatement ps =
            con.prepareStatement(sql);

        ResultSet rs =
            ps.executeQuery()
    ) {

        System.out.println("Product Details:");

        while (rs.next()) {

            int id = rs.getInt("ProductID");
            String name =
                rs.getString("ProductName");
            double price =
                rs.getDouble("Price");
            int quantity =
                rs.getInt("Quantity");

            System.out.println(
                id + " | " +
                name + " | " +
                price + " | " +
                quantity
            );
        }

    } catch (SQLException e) {

        System.out.println(
            "Database error while retrieving products."
        );

        System.out.println(

```

```

        "Error: " + e.getMessage()
    );
}
}
}

```

6. Explanation of the JDBC Connectivity Steps

The JDBC process can be represented as:

Step 1 → Load/Register JDBC Driver



Step 2 → Establish Database Connection



Step 3 → Create SQL Statement



Step 4 → Execute SQL Query



Step 5 → Process Result



Step 6 → Close Resources

Step 1 – Load JDBC Driver

Modern JDBC applications generally discover the driver automatically when the JDBC driver JAR is available.

For older JDBC code, the driver can explicitly be loaded:

```
Class.forName("com.mysql.cj.jdbc.Driver");
```

This is not normally required with current JDBC drivers.

Step 2 – Establish Connection

Connection con =

```
    DriverManager.getConnection(
        URL, USER, PASSWORD
    );
```

This establishes communication between Java and MySQL.

Example URL:

```
jdbc:mysql://localhost:3306/ecommerce
```

Step 3 – Create PreparedStatement

String sql =

```
"INSERT INTO Product " +
"(ProductID, ProductName, Price, Quantity) " +
"VALUES (?, ?, ?, ?)";
```

PreparedStatement ps =

```
    con.prepareStatement(sql);
```

PreparedStatement is preferred over constructing SQL using string concatenation because it supports parameter binding and helps protect against SQL injection.

Step 4 – Set Parameters

```
ps.setInt(1, id);
```

```
ps.setString(2, name);
```

```
ps.setDouble(3, price);
```

```
ps.setInt(4, quantity);
```

The ? placeholders are replaced with the supplied values.

Step 5 – Execute SQL

For INSERT, UPDATE or DELETE:

```
int rows = ps.executeUpdate();
```

For SELECT:

```
ResultSet rs = ps.executeQuery();
```

7. Retrieving Product Data

The SQL query is:

```
SELECT ProductID, ProductName, Price, Quantity  
FROM Product;
```

The result is stored in a ResultSet.

```
while (rs.next()) {
```

```
    int id = rs.getInt("ProductID");
```

```
    String name =
```

```
        rs.getString("ProductName");
```

```
    double price =
```

```
        rs.getDouble("Price");
```

```
    int quantity =
```

```
        rs.getInt("Quantity");
```

```
}
```

rs.next() moves the cursor to the next record.

8. Exception Handling

Database operations can fail because of:

- Incorrect database URL
- Database server not running
- Invalid username/password
- Table not found
- Duplicate primary key
- Invalid SQL syntax
- Network/database connection failure

These problems are handled using:

```
try {
```

```
    // JDBC operations
```

```
}
```

```
catch (SQLException e) {
```

```
    // Handle database error
```

```
}
```

Example:

```
catch (SQLException e) {
```

```
    System.out.println(
```

```
        "Database operation failed."
```

```
    );
```

```
    System.out.println(
```

```
        "Error: " + e.getMessage()
```

```
    );
```

```
}
```

For a production application, errors should be logged appropriately rather than exposing detailed database errors directly to customers.

9. Resource Management

JDBC resources should be closed after completing the operation.

The program uses try-with-resources:

```
try (  
    Connection con =  
        DriverManager.getConnection(URL, USER, PASSWORD);  
  
    PreparedStatement ps =  
        con.prepareStatement(sql);  
  
    ResultSet rs =  
        ps.executeQuery()  
) {  
    // Process result  
}
```

When the block finishes, Java automatically closes:

ResultSet
↓
PreparedStatement
↓
Connection

This reduces the possibility of connection leaks and improves application reliability.

10. INSERT and SELECT Operations

Insert

```
INSERT INTO Product  
VALUES (101, 'Laptop', 55000.00, 10);
```

Retrieve

```
SELECT * FROM Product;
```

Expected Output

ProductID	ProductName	Price	Quantity
101	Laptop	55000	10

11. Design Decision and Justification

For the proposed e-commerce application, PreparedStatement is selected instead of Statement.

Reasons:

- Supports parameterized SQL.
- Reduces SQL injection risk.
- Makes repeated queries easier to execute.
- Separates SQL structure from input values.

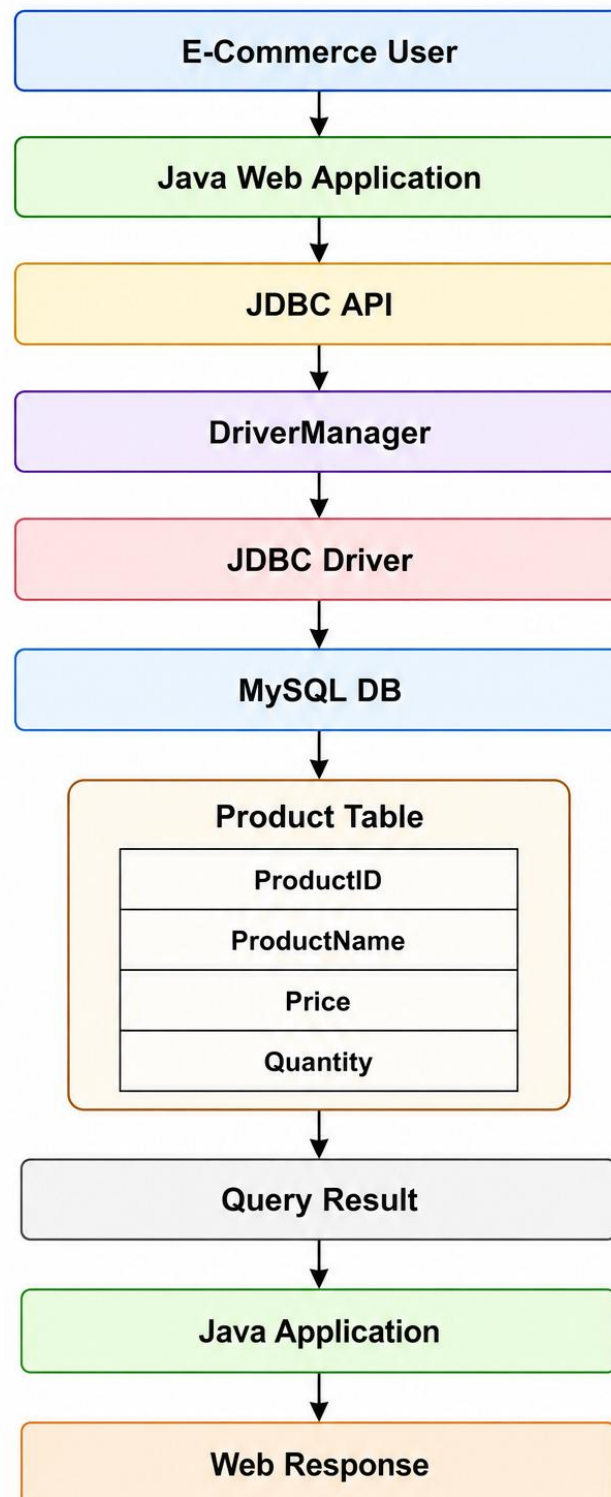
Similarly, try-with-resources is selected because it automatically releases JDBC resources, preventing connection leaks.

Thus, the design combines:

PreparedStatement
+
SQLException handling
+
try-with-resources
+

SQL queries
to produce a more secure, reliable, and maintainable JDBC application.

12. Complete JDBC Architecture and Processing Flow



13. Conclusion

JDBC provides a standard mechanism for a Java web application to communicate with a relational database. The proposed e-commerce system establishes a database connection, uses `PreparedStatement` to insert product data safely, uses `ResultSet` to retrieve products, and applies exception handling and try-with-resources for reliable resource management.